Should we make std::linalg reductions deduce return types like fold algorithms?

Document #:	P3809R0 [Latest] [Status]
Date:	2025-08-07
Project:	Programming Language C++
Audience:	Library Evolution

Reply-to:	Mark Hoemmen <mhoemmen@nvidia.com>

Contents

§ 1 Abstract

This proposal does not include normative changes; it is informational only.

C++26's `std::linalg::dot` and similar reduction-like algorithms return the same type as their initial value, just like C++17's `std::reduce`. In contrast, C++23's `std::ranges::fold_left`, `fold_right`, and `fold_left_with_iter` deduce their return type as "the decayed result of invoking the binary operation with `T` (the initial value) and the reference type of the range."

Some C++ Standard Committee members have asked that we change `std::linalg` to imitate `std::ranges::fold_left` instead of `std::reduce`. We discuss advantages and disadvantages of this change, and conclude that we should retain `std::linalg`'s current behavior.

§ 2 Introduction

The C++ Standard Committee voted the linear algebra proposal [P1673R13] into the C++26 working draft. This adds many algorithms to the new `std::linalg` namespace. Several of these algorithms behave like reductions, in that they apply a binary operator to the elements of one or two `mdspan` object(s) in order to reduce them to a single value. These *reduction-like* algorithms include

— the vector dot products `dot` and `dotc`,

— `vector_sum_of_squares`,

— the vector norms `vector_two_norm` and `vector_abs_sum`, and

— the matrix norms `matrix_frob_norm`, `matrix_one_norm`, and `matrix_inf_norm`.

All these algorithms take an optional initial value. Just like C++17 `std::reduce`, the return type of the algorithms is the same as the initial value's type. For example, if `x` is an `mdspan`, `std::linalg::vector_two_norm(x, 0.0)` returns `double`, regardless of `x`'s value type. If the value type is `double`, `std::linalg::vector_two_norm(x, 0)` would return `int`.

C++23 added the `fold_left`, `fold_right`, and `fold_left_with_iter` algorithms to `std::ranges`. All of them require an initial value of type `T`. However, their return type might not necessarily be `T`. Instead, the `fold_*` algorithms deduce the return type as "the decayed result of invoking the binary operation with `T` (the initial value) and the reference type of the range." For example, if `x` is a range of `double`, then `std::ranges::fold_left(x, 0)` would return `double`, even though the initial value is an `int`.

Some C++ Standard Committee members have asked that we change `std::linalg`'s reduction-like algorithms to imitate `std::ranges::fold_left` instead of `std::reduce`. Each approach has its advantages and disadvantages. We conclude that we should retain `std::linalg`'s current behavior. In our view, the current behavior makes sense for the linear algebra domain, even though it is not consistent with the various `fold_*` algorithms.

The same design concern applies to the numeric range algorithms proposed in [P3732R0]. Revision 0 of P3732 proposes that `std::ranges::reduce` deduce its return type in the same way as `std::ranges::fold_left`. We think that this is a reasonable design choice, even though we do not think `std::linalg`'s algorithms need to behave the same way.

§ 3 Arguments for and against the status quo

§ 3.1 What is the status quo and what is the suggested change?

The status quo is that for `std::linalg::dot` and all the other reduction-like algorithms in `std::linalg`, the return type is the same as the initial value type. For example, if `x` is an `mdspan`, `std::linalg::vector_two_norm(x, 0.0)` returns `double`, regardless of `x`'s value type. If the value type is `double`, `std::linalg::vector_two_norm(x, 0)` would return `int`.

The status quo is "imitate C++17's `std::reduce`." The change suggested by some C++ Standard Committee members is to imitate `fold_*` instead.

§ 3.2 Arguments for the status quo (imitate `std::reduce`)

1. It makes the return type obvious.

2. It works straightforwardly with expression templates, as we show later in this paper.

3. It is consistent with the BLAS Standard's Fortran 95 interface (where the initial value is an `INOUT` argument, so the initial value type is the same as the result type).

4. Users who are familiar with the linear algebra domain and with mixed-precision floating-point computations are more likely to state return and accumulator types explicitly, rather than rely on interfaces to "guess" them.

§ 3.3 Arguments against the status quo (imitate `fold_*`)

1. `dot(range_of_double, 0)` truncating the result by returning `int` is a usability pitfall.

2. The current interface may give users a false impression that they are controlling the precision of computation, because the initial value type may not be the same as the binary operator's return type.

3. If [P3732R0] (numeric range algorithms) is accepted, then `std::ranges::reduce` would likely have behavior consistent with C++23 `fold_*`. Making `std::linalg`'s reduction-like algorithms consistent with that would make it possible to implement `std::linalg` using numeric range algorithms.

§ 4 Affected algorithms in `std::linalg`

This issue affects the following algorithms in [linalg] with overloads that have `Scalar` or `sum_of_squares_result<Scalar>` return type. Overloads with `auto` return type deduce it from that of the overloads with `Scalar` return type.

— `dot` ([linalg.algs.blas1.dot])

— `dotc` ([linalg.algs.blas1.dot])

— `vector_sum_of_squares` ([linalg.algs.blas1.ssq]) (please see also the separate LWG issue)

— `vector_two_norm` ([linalg.algs.blas1.nrm2])

— `vector_abs_sum` ([linalg.algs.blas1.asum])

— `matrix_frob_norm` ([linalg.algs.blas1.matfrobnorm])

— `matrix_one_norm` ([linalg.algs.blas1.matonenorm])

— `matrix_inf_norm` ([linalg.algs.blas1.matinfnorm])

§ 5 Examples contrasting C++17 `reduce` and `fold_*`

The following examples of range and initial value types show differences between `std::reduce`'s results and `std::ranges::fold_*`'s results.

§ 5.1 Range of `float` with `uint64_t` initial value

The following example (Compiler Explorer link) sums a range of `float` with a nonzero `uint64_t` initial value using `std::reduce` and `std::ranges::fold_left`, and then compares the result with summing the same `float` range with a `double` initial value.

```
#include <algorithm>
#include <cstdint>
#include <limits>
#include <numeric>
#include <print>
#include <type_traits>
#include <vector>

int main() {
  // [0, 16777216] is the max contiguous range of
  // nonnegative integers that fit in float (FP32).
  const float big_fp32 = 16777215.0f;
  std::print("big_fp32 = {}\n", big_fp32);
  std::print("big_fp32 + 1.0f = {}\n", big_fp32 + 1.0f);
```

```
  std::print("big_fp32 + 2.0f = {}\n", big_fp32 + 2.0f);

  const std::size_t count = 4;
  std::vector<float> vec(count, big_fp32);
  std::print("vec = {}\n", vec);

  const std::uint64_t initial_value = 1u;
  const std::plus<> op{};

  auto reduce_result = std::reduce(
    vec.begin(), vec.end(), initial_value, op);
  static_assert(
    std::is_same_v<decltype(reduce_result), std::uint64_t>);
  std::print("reduce_result = {}\n", reduce_result);

  auto fold_result = std::ranges::fold_left(
    vec, initial_value, op);
  static_assert(
    std::is_same_v<decltype(fold_result), float>);
  std::print("fold_result = {}\n", fold_result);

  std::vector<double> vec_fp64(count,
    static_cast<double>(big_fp32)
  );
  auto fold_fp64_result = std::ranges::fold_left(
    vec_fp64, static_cast<double>(initial_value), op);
  std::print("vec_fp64 = {}\n", vec_fp64);
  static_assert(
    std::is_same_v<decltype(fold_fp64_result), double>);
  std::print("fold_fp64_result = {}\n", fold_fp64_result);

  return 0;
}
```

It prints the following output.

```
big_fp32 = 16777215
big_fp32 + 1.0f = 16777216
big_fp32 + 2.0f = 16777216
vec = [16777215, 16777215, 16777215, 16777215]
reduce_result = 67108861
fold_result = 67108864
vec_fp64 = [16777215, 16777215, 16777215, 16777215]
fold_fp64_result = 67108861
```

Why would users try summing a ‘float’ range with a `uint64_t` initial value? In this example, we know that

— each value is a nonnegative integer that fits exactly in both `float` and in `uint32_t`, but

— `float` cannot represent the sum of the values exactly.

Novice users might reasonably expect that using a `uint64_t` as the initial value would turn the `float`s into integers and thus would perform the sum exactly. Alas, they would be

wrong. For both algorithms, each binary `operator+` invocation first converts the `uint64_t` term to `float`, and then does the computation in `float` precision. C++ itself does the "wrong thing" here; it's not really either algorithm's "fault."

The difference between algorithms is in return types and order of operations. `std::reduce` assigns the `float` binary operation result to a `uint64_t` accumulator and then returns `uint64_t`, thus concealing the loss of information. `std::ranges::fold_left` returns `float`, which indicates that something bad may have happened.

In this case, `std::reduce` accidentally got the right answer, probably because the initial value of one got added at the end (as the numeric algorithms have permission to reorder terms).

§ 5.2 Mixed floating-point types with more precise addition

This situation simulates the increasing variety of number types designed for the performance needs of machine learning.

Let `float_float` represent a 64-bit number type that uses the "double-double" approach to approximate `double`-precision floating-point arithmetic.

```
// for resolving ambiguities in overloaded operator+
struct float_base {};

struct float_float : float_base {
  float_float() = default;
  float_float(float);

  // ... other member functions ...

  friend float_float operator+(float_float, float_float);
  template<class T>
    requires(
      std::is_base_of_v<float_base, T> &&
      ! std::is_same_v<T, float_float>)
  friend double operator+(float_float, T) {
    // ... implementation ...
  }
};
```

Let `binary48` represent a 6-byte type that has more significand and exponent bits than IEEE 754 `binary32` (`float`), but fewer than `binary64` (`double`). For example, `binary48` might have 9 exponent bits, 38 significand bits, and 1 sign bit.

```
struct binary48 : float_base {
  binary48() = default;
  binary48(float);

  friend binary48 operator+(binary48, binary48);
  template<class T>
```

```
    requires(
      std::is_base_of_v<float_base, T> &&
      ! std::is_same_v<T, binary48>)
  friend double operator+(binary48, T) {
    // ... implementation ...
  }
};
```

Both types have implicit conversions from `float`, but are not mutually convertible. Overloaded `operator+` between either of the types results in `double` (a type that can represent all the values of either).

A `float_float` object can represent more values than `binary48`, so users might try to sum their range of `binary48` into a `float_float` accumulator. They *should* use a `double` accumulator in this case, because this is the return type of `operator+`. However, users don't always do what we expect!

In this case, `std::ranges::fold_left` does the right thing. It would accumulate into a `double` and return `double`. In contrast, `std::reduce` with `float_float` initial value would cast each intermediate `double` result to `float_float`. This may lose information.

The last section of this paper shows this example in full.

§ 6 Expression templates complicate use of `fold_*`

The `fold_*` algorithms deduce the return type as "the decayed result of invoking the binary operation with `T` (the initial value) and the reference type of the range." This can complicate use of range or initial value types where the binary operation returns an expression template. Simple `fold_*` invocations with `std::plus<>` may fail to compile, for example. In contrast, C++17 `std::reduce` has no problem as long as the initial value type is not an expression template.

We do not actually want the `fold_*` algorithms to return an expression template. An expression template encodes a deferred computation. Thus, it may hold a reference or pointer to some other variables that may live on the stack. As a result, returning a proxy reference from a function carries the risk of dangling memory.

At the same time, we want to permit use of expression templates, because they are common operations for number-like types where creating and returning temporary values can have a high cost. Using expression templates for arithmetic on such types can reduce dynamic memory allocation or stack space requirements. Examples include arbitrary-precision floating-point numbers, or *ensemble* data types that help sample a physics simulation over uncertain input data, by propagating multiple samples together as if they are a single floating-point number (see e.g., [Phipps 2017]).

The last section of this papes gives an example of an expression templates system for a "fixed-width vector" type `fixed_vector<ValueType, Size>`. The two template parameters correspond to those of `std::array`. Its behavior is like that of `std::array`, and in addition, overloaded arithmetic operators implement a vector algebra. This algebra consists of vectors – either `fixed_vector` or an expression template thereof – and scalars (any type `T` not a vector, for which `operator*(T, ValueType)` is well formed).

We show some use cases below. All of them reduce over the elements of `vec`, which has three `fixed_vector<float, 4>` values. The `fixed_vector` value `zero` has four zeros, and serves as the initial value for all the reductions. The `fixed_vector` value `expected_result` is the expected result for all the reductions.

```
using expr::fixed_vector;

std::vector<fixed_vector<float, 4>> vec{
  fixed_vector{std::array{1.0f, 2.0f, 3.0f, 4.0f}},
  fixed_vector{std::array{0.5f, 1.0f, 1.5f, 2.0f}},
  fixed_vector{std::array{0.25f, 0.5f, 0.75f, 1.0f}}
};
const fixed_vector zero{std::array{0.0f, 0.0f, 0.0f, 0.0f}};
const fixed_vector expected_result{std::array{
  1.75f, 3.5f, 5.25f, 7.0f
}};
```

Invoking `std::reduce` with `std::plus<>` as the binary operator and a `fixed_vector<float, 4>` instance as the initial value both compiles and runs correctly. It's obvious to readers what this means. It's even possible to omit both the binary operator and the initial value, since the defaults for those would be `std::plus<>` and `fixed_vector<float, 4>{}`, respectively. The call operator of `std::plus<>` accepts any two types (possibly even heterogeneous types, though they both decay to `fixed_vector<float, 4>` in this case) and returns an expression template type (`binary_plus_expr`). `std::reduce` assigns that result to the accumulator, thus invoking `fixed_vector`'s templated assignment operator that works with any fixed-vector-like type. Using `std::reduce` with `std::plus<>` thus achieves the desired effect of expression templates: avoiding unnecessary temporaries.

```
auto reduce_result = std::reduce(
  vec.begin(), vec.end(), zero, std::plus<>{}
);
for (std::size_t k = 0; k < 4; ++k) {
  assert(reduce_result[k] == expected_result[k]);
}
```

Invoking `std::ranges::fold_left` with `std::plus<>` as the binary operator does *not* compile, because the return type of `std::plus<>::operator()` is an expression template type (`binary_plus_expr`). This is a feature, not a bug, because returning an expression template here would necessarily dangle. The compilation error manifests as failure to satisfy *`indirectly-binary-left-foldable-impl`*, because `binary_plus_expr` is not `movable`. It's not `movable` because its copy assignment operator is implicitly deleted, because it has reference members. That's good! The correct design for an expression template is

that it is not copy-assignable. Note, however, that nothing prevents users from writing an expression template that stores pointers instead of references. That could result in a `movable` type, which would, in turn, cause dangling if used in `fold_left` with `std::plus<>`.

Invoking `fold_left` with the binary operator spelled out as `std::plus<fixed_vector<float, 4>>` compiles and runs correctly. However, this may lose the advantage of expression templates. For each element of the `std::vector`, the algorithm's invocation of the binary operator returns a new temporary `fixed_vector`, assigns it to the accumulator, and then discards the temporary. Expression templates were designed to make this temporary unnecessary.

The Standard lets compilers perform copy elision ([class.copy.elision]) for unnamed return value cases like this. That would effectively eliminate the temporary. However, copy elision is not mandatory in this case.

```
auto fold_result = std::ranges::fold_left(
  vec.begin(), vec.end(),
  zero,
  std::plus<fixed_vector<float, 4>>{}
);
for (std::size_t k = 0; k < 4; ++k) {
  assert(fold_result[k] == expected_result[k]);
}
```

Another `fold_left` approach is to use a lambda returning `fixed_vector` as the binary operator. As with `std::plus<fixed_vector<float, 4>>`, this depends on copy elision in order to eliminate the temporary. The lambda also makes the code harder to read and write.

```
// fold_left with a lambda must return fixed_vector
// so that the algorithm can deduce the result type.
auto fold_lambda_result = std::ranges::fold_left(
  vec.begin(), vec.end(),
  zero,
  [] (const auto& x, const auto& y) {
    return fixed_vector<float, 4>(x + y);
  }
);
for (std::size_t k = 0; k < 4; ++k) {
  assert(fold_lambda_result[k] == expected_result[k]);
}
```

In summary,

- `std::reduce` with expression templates Just Works; while
- `std::ranges::fold_left`
  - does not compile for the default `std::plus<>` case, and
  - depends on non-mandatory copy elision to avoid temporaries with expression templates.

§ 7 vector_sum_of_squares

LWG Issue 4302 shows a separate design issue with `vector_sum_of_squares`. If the C++ Standard Committee resolves this issue by removing `vector_sum_of_squares` entirely, then we won't have to worry about its return type.

§ 8 Conclusions

The status quo for return types of `std::linalg`'s reduction-like algorithms is to imitate C++17 `std::reduce`. Some have suggested changing this to imitate C++23 `std::ranges::fold_left` and related algorithms. Given the disadvantages of `fold_left`'s approach for use cases of importance to `std::linalg`'s audience, we recommend against changing `std::linalg` at this point.

§ 9 Examples: Code links and listings

§ 9.1 float-float and 6-byte float

This example shows the difference in return type between `std::ranges::fold_left` and `std::reduce` for mixed-precision arithmetic that returns a type of a higher precision. Here is a Compiler Explorer link.

```
#include <cassert>
#include <cstdint>
#include <type_traits>
#include <utility>

// We define 2 noncomparable floating-point types,
// both larger than float and smaller than double:
// float_float and binary48.

// for resolving ambiguities in overloaded operator+
struct float_base {};

struct float_float;
struct binary48;

// float_float uses the "double-double" technique
// (storing a pair of floating-point values
// to represent a sum x_small + factor * x_large)
// but with float instead of double.
// The result takes 64 bits,
// but has less exponent range than double.
// One would do this to simulate double-precision arithmetic
// on hardware where that arithmetic is slow or not supported.
//
// This class doesn't actually implement correct arithmetic;
// the point is just to show that this compiles.
struct float_float : float_base {
```

```
  float_float() = default;
  float_float(float) {}

  friend float_float operator+(float_float, float_float) {
    return {};
  }
  template<class T>
    requires(
      std::is_base_of_v<float_base, T> &&
      ! std::is_same_v<T, float_float>)
  friend double operator+(float_float, T) {
    return 0.0;
  }
};

// binary48 represents a 6-byte floating-point type
// that has more significand and exponent bits than
// IEEE 754 binary32 (float),
// but fewer than binary64 (double).
// For example, binary48 might have 9 exponent bits,
// 38 significand bits, and 1 sign bit.
//
// We don't actually implement correct arithmetic;
// the point is just to show that this compiles.
struct binary48 : float_base {
  binary48() = default;
  binary48(float);

  friend binary48 operator+(binary48, binary48) {
    return {};
  }
  template<class T>
    requires(
      std::is_base_of_v<float_base, T> &&
      ! std::is_same_v<T, binary48>)
  friend double operator+(binary48, T) {
    return 0.0;
  }
};

// This is well-formed if and only if
// the return type of operator+ called on (T, ValueType)
// is the same as T.
template<
  class /* initial value */ T,
  class /* type of each element of the range */ ValueType>
constexpr bool test() {
  return
    std::is_same_v<
      decltype(
        std::declval<T>() +
        std::declval<ValueType>()),
      T
    >;
}
```

```
int main() {
  {
    static_assert(! test<std::size_t, float>());
    static_assert(test<float, std::size_t>());

    static_assert(! test<int32_t, uint64_t>());
    static_assert(test<uint64_t, int32_t>());

    static_assert(! test<std::size_t, double>());
    static_assert(test<double, std::size_t>());

    static_assert(! test<uint64_t, double>());
    static_assert(test<double, uint64_t>());

    // Make sure that our custom number types
    // have non-ambiguous overloaded operator+
    float_float x;
    static_assert(std::is_same_v<decltype(x + x), float_float>);

    binary48 y;
    static_assert(std::is_same_v<decltype(y + y), binary48>);

    static_assert(std::is_same_v<decltype(x + y), double>);
    static_assert(std::is_same_v<decltype(y + x), double>);

    static_assert(! test<double, float_float>());
    static_assert(! test<double, binary48>());
    static_assert(! test<long double, binary48>());

    // This is the bad thing
    static_assert(! test<float_float, binary48>());
  }

  return 0;
}
```

§ 9.2 Expression templates and `fold_left` and `reduce` return types

This example shows how to use `std::ranges::fold_left` and `std::reduce` when the return type of `operator+` is an expression template. Here is a Compiler Explorer link.

```
#include <algorithm>
#include <array>
#include <cassert>
#include <cstdint>
#include <numeric>
#include <type_traits>
#include <utility>
#include <vector>
#include <version>

// Expression templates
```

```cpp
namespace expr {

namespace impl {

template<std::size_t Size>
struct fixed_vector_expr {
  static constexpr std::size_t size() { return Size; }
};

template<std::size_t Size0, std::size_t Size1>
using binary_fixed_vector_expr_t = fixed_vector_expr<
  Size0 < Size1 ? Size1 : Size0
>;

} // namespace impl

template<class T>
constexpr bool is_fixed_vector_v = false;

template<class X>
  requires(is_fixed_vector_v<X>)
struct unary_plus_expr : public impl::fixed_vector_expr<X::size()> {
  constexpr auto operator[] (std::size_t k) const {
    return +x[k];
  }

  const X& x;
};

template<class X>
constexpr bool is_fixed_vector_v<unary_plus_expr<X>> = true;

template<class Left, class Right>
  requires(is_fixed_vector_v<Left> && is_fixed_vector_v<Right>)
class binary_plus_expr :
  public impl::binary_fixed_vector_expr_t<Left::size(), Right::size()>
{
public:
  // Constructor needed due to inheritance;
  // otherwise this class could be an aggregate.
  binary_plus_expr(const Left& lt, const Right& rt) : left(lt), right(rt)
        {}

  constexpr auto operator[] (std::size_t k) const {
    return left[k] + right[k];
  }

private:
  const Left& left;
  const Right& right;
};

template<class L, class R>
constexpr bool is_fixed_vector_v<binary_plus_expr<L, R>> = true;
```

```
template<class X>
  requires(is_fixed_vector_v<X>)
struct unary_minus_expr :
  public impl::fixed_vector_expr<X::size()>
{
  constexpr auto operator[] (std::size_t k) const {
    return -x[k];
  }

  const X& x;
};

template<class X>
constexpr bool is_fixed_vector_v<unary_minus_expr<X>> = true;

template<class Left, class Right>
  requires(is_fixed_vector_v<Left> && is_fixed_vector_v<Right>)
class binary_minus_expr :
  public impl::binary_fixed_vector_expr_t<Left::size(), Right::size()>
{
public:
  // Constructor needed due to inheritance;
  // otherwise this class could be an aggregate.
    binary_minus_expr(const  Left&  lt,  const  Right&  rt)  :  left(lt),
        right(rt) {}

  constexpr auto operator[] (std::size_t k) const {
    return left[k] - right[k];
  }

private:
  const Left& left;
  const Right& right;
};

template<class L, class R>
constexpr bool is_fixed_vector_v<binary_minus_expr<L, R>> = true;

template<class ScalingFactor, class Vector>
  requires(
    ! is_fixed_vector_v<ScalingFactor> &&
    is_fixed_vector_v<Vector>)
struct scale_expr :
  public impl::fixed_vector_expr<Vector::size()>
{
public:
  // Constructor needed due to inheritance;
  // otherwise this class could be an aggregate.
  scale_expr(const ScalingFactor& sf, const Vector& v) :
    alpha(sf), vec(v)
  {}

  constexpr auto operator[] (std::size_t k) const {
    return alpha * vec[k];
  }
```

```
private:
  const ScalingFactor& alpha;
  const Vector& vec;
};

template<class S, class V>
constexpr bool is_fixed_vector_v<scale_expr<S, V>> = true;

template<class ValueType, std::size_t Size>
class fixed_vector;

template<class ValueType, std::size_t Size>
constexpr bool
is_fixed_vector_v<fixed_vector<ValueType, Size>> = true;

template<class ValueType, std::size_t Size>
class fixed_vector : public impl::fixed_vector_expr<Size> {
public:
  constexpr fixed_vector() = default;

  constexpr fixed_vector(const std::array<ValueType, Size>& data)
    : data_(data)
  {}

  template<class Expr>
  constexpr fixed_vector(const Expr& expr) {
    for (std::size_t k = 0; k < Size; ++k) {
      data_[k] = expr[k];
    }
  }

  template<class Expr>
  constexpr fixed_vector& operator=(const Expr& expr) {
    for (std::size_t k = 0; k < Size; ++k) {
      data_[k] = expr[k];
    }
    return *this;
  }

  template<class Expr>
  constexpr fixed_vector& operator+=(const Expr& expr) {
    for (std::size_t k = 0; k < Size; ++k) {
      data_[k] += expr[k];
    }
    return *this;
  }

  static constexpr std::size_t size() { return Size; }

  constexpr ValueType operator[] (std::size_t k) const {
    return data_[k];
  }

  constexpr ValueType& operator[] (std::size_t k) {
    return data_[k];
```

```
  }

private:
  std::array<ValueType, Size> data_{};
};

template<class X>
  requires(is_fixed_vector_v<X>)
constexpr unary_plus_expr<X>
operator+(const X& x) {
  return {x};
}

template<class X>
  requires(is_fixed_vector_v<X>)
constexpr unary_minus_expr<X>
operator-(const X& x) {
  return {x};
}

template<class L, class R>
  requires(
    is_fixed_vector_v<L> &&
    is_fixed_vector_v<R>)
constexpr binary_plus_expr<L, R>
operator+(const L& left, const R& right) {
  return {left, right};
}

template<class L, class R>
  requires(
    is_fixed_vector_v<L> &&
    is_fixed_vector_v<R>)
constexpr binary_minus_expr<L, R>
operator-(const L& left, const R& right) {
  return {left, right};
}

template<class ScalingFactor, class Vector>
  requires(
    ! is_fixed_vector_v<ScalingFactor> &&
    is_fixed_vector_v<Vector>)
constexpr scale_expr<ScalingFactor, Vector>
operator*(const ScalingFactor& alpha, const Vector& v) {
  return {alpha, v};
}

} // namespace expr

int main() {
  {
    using expr::fixed_vector;

    // Test our expression templates system.
    {
```

```
  fixed_vector x{std::array{1.0f, 2.0f, 3.0f, 4.0f}};
  fixed_vector y{std::array{-1.0f, -2.0f, -3.0f, -4.0f}};

  fixed_vector<float, 4> x_plus_y = x + y;
  fixed_vector x_plus_y_expected{std::array{0.0f, 0.0f, 0.0f, 0.0f}};
  for (std::size_t k = 0; k < 4; ++k) {
    assert(x_plus_y[k] == x_plus_y_expected[k]);
  }

  fixed_vector<float, 4> three_times_x = 3.0f * x;
     fixed_vector  three_times_x_expected{std::array{3.0f,  6.0f,  9.0f,
    12.0f}};
  for (std::size_t k = 0; k < 4; ++k) {
    assert(three_times_x[k] == three_times_x_expected[k]);
  }

  fixed_vector<float, 4> z = 3.0f * x + y;
  fixed_vector z_expected{std::array{2.0f, 4.0f, 6.0f, 8.0f}};
  for (std::size_t k = 0; k < 4; ++k) {
    assert(z[k] == z_expected[k]);
  }
}

std::vector<fixed_vector<float, 4>> vec{
  fixed_vector{std::array{1.0f, 2.0f, 3.0f, 4.0f}},
  fixed_vector{std::array{0.5f, 1.0f, 1.5f, 2.0f}},
  fixed_vector{std::array{0.25f, 0.5f, 0.75f, 1.0f}}
};
const fixed_vector zero{std::array{0.0f, 0.0f, 0.0f, 0.0f}};
const fixed_vector expected_result{std::array{
  1.75f, 3.5f, 5.25f, 7.0f
}};

// Show that a manual for loop with std::plus<> produces
// the expected result.  std::plus<>::operator() returns
// an expression template (binary_plus_expr) here.
{
  fixed_vector<float, 4> plus_result;
  std::plus<> plus{};
  static_assert(
    std::is_same_v<
      decltype(plus(plus_result, vec[0])),
      expr::binary_plus_expr<
        fixed_vector<float, 4>,
        fixed_vector<float, 4>>
    >
  );

  for (std::size_t k = 0; k < vec.size(); ++k) {
    plus_result = plus(plus_result, vec[k]);
  }
  for (std::size_t k = 0; k < 4; ++k) {
    assert(plus_result[k] == expected_result[k]);
  }
}
```

```
    // Show that a manual for loop produces the expected result.
    // This invokes fixed_vector's generic operator+=
    // that works for any "fixed-vector-like" type, including
    // fixed_vector itself and any of its expression templates.
    {
      fixed_vector<float, 4> manual_result;
       for (std::size_t outer_index = 0; outer_index < vec.size(); ++out-
        er_index) {
        manual_result += vec[outer_index];
      }
      for (std::size_t k = 0; k < 4; ++k) {
        assert(manual_result[k] == expected_result[k]);
      }
    }

#if defined(__cpp_lib_ranges_fold)
    // fold_left with std::plus<> does NOT compile,
    // because the return type of std::plus<>::operator()
    // is an expression template type (binary_plus_expr).
    // This is a feature, not a bug, because returning an
    // expression template here would necessarily dangle.
    // This manifests as failure to satisfy
    // _`indirectly-binary-left-foldable-impl`_,
    // because binary_plus_expr is not movable
    // (because its copy assignment operator is implicitly deleted,
    // because it has reference members).
    auto fold_result = std::ranges::fold_left(
      vec.begin(), vec.end(),
      zero,
      //std::plus<>{}
      std::plus<fixed_vector<float, 4>>{}
    );
    for (std::size_t k = 0; k < 4; ++k) {
      assert(fold_result[k] == expected_result[k]);
    }

    // fold_left with a lambda must return fixed_vector
    // so that the algorithm can deduce the result type.
    auto fold_lambda_result = std::ranges::fold_left(
      vec.begin(), vec.end(),
      zero,
      [] (const auto& x, const auto& y) {
        return fixed_vector<float, 4>(x + y);
      }
    );
    for (std::size_t k = 0; k < 4; ++k) {
      assert(fold_lambda_result[k] == expected_result[k]);
    }
#endif

    // std::reduce with std::plus<> compiles and runs correctly,
    // because the initial value type is fixed_vector
    // and std::reduce casts the expression template type
    // (binary_plus_expr) returned from std::plus<>::operator()
    // to the initial value type.
    auto reduce_result = std::reduce(
```

```
      vec.begin(), vec.end(), zero, std::plus<>{}
    );
    for (std::size_t k = 0; k < 4; ++k) {
      assert(reduce_result[k] == expected_result[k]);
    }
  }

  return 0;
}
```

§ 10 References

[P1673R13] Mark Hoemmen, Daisy Hollman,Christian Trott,Daniel Sunderland,Nevin Liber,Alicia KlinvexLi-Ta Lo,Damien Lebrun-Grandie,Graham Lopez,Peter Caday,Sarah Knepper,Piotr Luszczek,Timothy Costa. 2023-12-18. A free function linear algebra interface based on the BLAS.

https://wg21.link/p1673r13

[P3732R0] Ruslan Arutyunyan, Mark Hoemmen, Alexey Kukanov, Bryce Adelstein Lelbach, Abhilash Majumder. 2025-06-27. Numeric Range Algorithms.

https://wg21.link/p3732r0

[Phipps 2017] E. Phipps, M. D'Elia, H. C. Edwards, M. Hoemmen, and others. "Embedded Ensemble Propagation for Improving Performance, Portability, and Scalability of Uncertainty Quantification on Emerging Computational Architectures," *SIAM Journal on Scientific Computing*, 39(2), pp. 162 - 193, 2017.

https://doi.org/10.1137/15M1044679